



SENIOR PROJECT REPORT



Submitted By

Ayah Samer Said

20221501027

Project Supervisor

Dr. Amani Abu Jabal

Senior Project

Department of Computer Science

School of Computing

German Jordanian University

First Semester 2025/2026

AUTHORIZATION FORM

I, Ayah Samer Said, authorize German Jordanian University to supply copies of my Graduation Project Report to libraries, establishments or individuals on request, according to the regulations of German Jordanian University.

Signature:

A handwritten signature in black ink, appearing to read 'Ayah', with a stylized flourish extending to the right.

Date:

1/1/2026

ACKNOWLEDGEMENTS

First and foremost, I am sincerely thankful to Allah Almighty for granting me the strength, patience, and guidance needed to complete this senior project. Without His mercy and countless blessings, reaching this milestone would not have been possible. I am deeply grateful for the resilience and wisdom He granted me throughout this journey.

My sincere gratitude is extended to Dr. Amani Abu Jabal, whose supervision and guidance were instrumental to the successful completion of this senior project. Her continuous support and insightful feedback greatly influenced the direction of this work and enhanced my technical and problem-solving skills.

I am deeply grateful to my parents and family, whose love and continuous encouragement gave me the motivation to overcome challenges. Their belief in me and confidence in my abilities were a steady source of strength throughout my academic journey.

I am thankful to my friends for their encouragement, patience, and support throughout this journey. Their shared experiences and constant motivation made the challenges easier to overcome and contributed to a positive and rewarding academic experience.

Lastly, I would like to extend my appreciation to the faculty members and examiners for their time and effort in reviewing this senior project. I am also grateful for their guidance, support, and dedication throughout my academic studies, which greatly enriched my learning experience.

Intelligent Weather-Based Outfit Recommendation

By

Ayah Samer Said

Supervisor

Dr.Amani Abu Jabal

ABSTRACT

The unpredictable nature of environmental conditions presents a challenge for individuals seeking to balance personal comfort, health considerations, and activity-specific dress codes. This project presents FitCast, a mobile application designed to bridge the gap between weather forecasting and personalized outfit recommendation.

Developed using Flutter and Firebase, the application features a custom-trained Random Forest classifier to predict optimal clothing layers based on environmental variables. The system integrates a reinforcement learning feedback loop that adapts to unique user thermal sensitivities over time, managed by an activity-aware outfit engine that resolves environmental and social constraints.

Unlike existing applications such as ArabiaWeather (طقس العرب) and Weather Fit, which primarily provide general forecasts or generic clothing visuals with limited personalization, FitCast emphasizes intelligent, context-aware recommendations tailored to user preferences and real-life scenarios. This project highlights how intelligent, data-based modeling can enhance personal utility and support more informed daily decisions.

TABLE OF CONTENTS

AUTHORIZATION FORM.....	II
ACKNOWLEDGEMENTS	III
ABSTRACT.....	IV
TABLE OF CONTENTS.....	V
TABLE OF FIGURES	VII
TABLE OF LISTINGS	VIII
1. CHAPTER I:.....	1
Introduction	1
1.1. Background	1
1.2. Motivation.....	2
1.3. Problem Statement	2
1.4. Research Objectives.....	3
1.5. Project Scope	4
1.6. Organization of the Report	4
2. CHAPTER II	5
System Analysis and Design.....	5
2.1. System Overview	5
2.2. System Architecture.....	5
2.2.1. Architectural Style.....	6
2.2.2. Core System Components.....	6
2.2.3. System Architecture Visualization	7
2.3. Functional and Non-Functional Requirements	8
2.3.1. Functional Requirements	8
2.3.2. Non-Functional Requirements	9

2.3.3.	System Use Case Diagram.....	10
2.3.4.	System Sequence Diagram.....	11
2.4.	Predictive Thermal Model.....	12
2.4.1.	Bioclimatic Feature Engineering	12
2.4.2.	Random Forest Classification.....	13
2.5.	Rule-Based Outfit Engine	14
2.5.1.	Context-Aware Activity Filtering	14
2.5.2.	Style Compatibility and Anchoring.....	15
2.5.3.	Personalization and Health Integration	15
3.	CHAPTER III.....	17
	System Implementation	17
3.1.	Development Environment.....	17
3.1.1.	Technology Stack.....	17
3.1.2.	Visual Asset Design.....	18
3.2.	External API Integrations	19
3.2.1.	Meteorological Data (OpenWeather API).....	19
3.2.2.	Intent Extraction and Validation (OpenAI ChatGPT API)	20
3.3.	Internal Machine Learning Service (FastAPI)	22
3.3.1.	Feature Engineering and Model Training	22
3.3.2.	Model Deployment (FastAPI Endpoint)	23
3.3.3.	Client-Side Integration (Flutter)	23
3.4.	Mobile Application Implementation.....	24
4.	CHAPTER IV.....	28
	EXPERIMENTAL RESULTS AND EVALUATION.....	28
4.1.	Introduction	28
4.2.	User Interface Implementation	28
4.2.1.	Authentication and Account Access	29
4.2.2.	Onboarding and Profile Customization.....	30
4.2.3.	Main Dashboard and User Interaction.....	31
4.2.4.	Dynamic Outfit Visualization and Layer Control	32
4.3.	System Scenarios and Case Studies	34
4.4.	Implementation Challenges and Solutions	36
5.	CHAPTER V.....	39
	CONCLUSION AND FUTURE WORK.....	39
5.1.	Conclusion.....	39
5.2.	Future Work.....	40
	REFERENCES	41

TABLE OF FIGURES

<i>Figure 2.1: High-level system architecture of FitCast illustrating the interaction between the mobile client, external APIs, and the data-driven intelligence pipeline.</i>	<i>7</i>
<i>Figure 2. .2: Use Case Diagram illustrating the primary interactions between the user, the FitCast system, and external APIs (OpenAI and OpenWeather).</i>	<i>10</i>
<i>Figure 2.3: Sequence diagram illustrating the conversational data collection and the hybrid processing flow, where the mobile client orchestrates external services (NLU, Weather) and the internal logic engine.</i>	<i>11</i>
<i>Figure 3.1 illustrates the core logic used to compute perceived temperature during dataset generation.</i>	<i>22</i>
<i>Figure 3.2 FastAPI inference endpoint used to predict the clothing layer from processed weather features.</i>	<i>23</i>
<i>Figure 3.3 Flutter service sending feature data to the FastAPI prediction endpoint.</i>	<i>24</i>
<i>Figure 4.1: FitCast sign-in and sign-up interface used for secure user authentication.</i>	<i>29</i>
<i>Figure4.2 : Sunny assistant collecting user thermal sensitivity preferences during the initial profile setup.</i>	<i>30</i>
<i>Figure 4.3: Main dashboard with dynamic weather visualization and avatar-based outfit interaction.</i>	<i>31</i>
<i>Figure 4.4: Outfit visualization screen displaying the outerwear layer, including jackets and accessories, with layer control arrows for navigating between clothing layers.</i>	<i>32</i>
<i>Figure 4.5: Thermal feedback interface allowing users to rate the recommended outfit as too cold, comfortable, or too warm for adaptive personalization.</i>	<i>33</i>

TABLE OF LISTINGS

<i>Listing 3-1: Core weather feature extraction and wind speed stabilization logic.</i>	<i>19</i>
<i>Listing 3-2: Core prompt logic used for activity classification in the NLU layer</i>	<i>21</i>
<i>Listing 3-3: Implementation of the anchoring logic used to select compatible outfit combinations.</i>	<i>25</i>

CHAPTER I: INTRODUCTION

1.1. Background

Outfit selection is often seen as a simple part of everyday life. However, when clothing choices are made based on misunderstood or changing weather conditions, they can cause discomfort that lasts throughout the day. This shows that even small daily decisions, such as what to wear, can have a noticeable impact on personal comfort, health, and overall well-being when weather factors are not properly considered.

Although weather forecasts are easily accessible, they are often shown as numerical values such as temperature, wind speed, and humidity. Users must interpret this information on their own to decide what to wear or how many layers are needed. This can lead to confusion, wasted time, and clothing choices that do not match the actual weather conditions.

In today's digital era, mobile applications play an important role in simplifying everyday tasks and supporting user convenience. While smartphones are widely used to manage daily routines, many existing weather-based applications focus mainly on presenting data rather than offering practical guidance. This gap highlights the need for intelligent systems that go beyond raw information and provide user-centered recommendations that improve comfort, preparedness, and efficiency in daily life.

1.2. Motivation

The motivation for this project comes from a personal interest in daily outfit planning and the lack of personalized guidance in existing platforms. While following social media accounts that offer weather-based outfit suggestions, it became clear that most recommendations are general, non-interactive, and not adapted to individual user preferences.

After noticing this issue, existing mobile applications with similar goals were reviewed. Most of them were found to rely on minimal user input and provide general suggestions. This made the idea of a more interactive system more appealing, where users can influence the recommendations through their input and feedback. Conversations with potential users further confirmed the usefulness of such a system, leading to the development of FitCast.

1.3. Problem Statement

Despite the wide availability of weather forecasting tools and weather-based outfit recommendation applications, many existing solutions offer limited practical value. The main issue is not the lack of environmental data, but the difficulty these systems have in converting this information into personalized, context-aware, and culturally appropriate clothing recommendations.

- **Failure to Account for Individual Thermal Sensitivity:**

Most existing applications assume that users react to weather conditions in the same way. This ignores individual differences in how people feel and tolerate temperature, which often leads to outfit recommendations that are uncomfortable or impractical for many users.

- **Lack of Contextual and Personal Integration:**

Existing applications usually collect only basic user information, such as name or gender, and provide static outfit recommendations. These recommendations often fail to consider personal preferences, specific activities, or changes in daily context.

- **Disregard for Cultural and Social Norms:**

In regions such as the Arab world, clothing choices are influenced by cultural and modesty requirements; however, many applications suggest attire that may be socially unsuitable, particularly in warm weather.

These limitations highlight the need for an intelligent and adaptive system that combines weather conditions with individual thermal sensitivity, contextual factors, and cultural considerations in order to provide realistic, personalized, and socially appropriate outfit recommendations.

1.4. Research Objectives

The main objectives of this project are to:

- Design and develop a mobile application that translates real-time weather data into personalized outfit recommendations tailored to individual users.
- Implement a predictive thermal model using a custom-trained Random Forest classifier to estimate perceived temperature and optimal clothing layers.
- Integrate a context-aware rule-based outfit engine that applies environmental, activity-related, and social constraints to ensure realistic and appropriate clothing suggestions.

- Incorporate an adaptive learning mechanism that utilizes user feedback to model individual thermal sensitivity and continuously refine recommendation accuracy over time.
- Develop an interactive and visually engaging user interface, featuring dynamic weather-based backgrounds and a generative avatar system synchronized with real-time conditions.

1.5. Project Scope

This project focuses on developing a cross-platform mobile application for Android and iOS that converts real-time weather data into functional, layering-based outfit recommendations. The scope includes a cloud-connected backend that adapts based on user feedback and a user interface designed for efficient daily use.

To ensure feasibility within the project timeline, certain functionalities are considered outside the scope of this work. The application is intended as a utility for physical comfort and preparedness and does not provide aesthetic fashion styling, color coordination, trend analysis, or professional fashion advice. The project prioritizes functional outfit guidance derived from environmental conditions rather than subjective style recommendations.

1.6. Organization of the Report

This report is structured into five chapters. Chapter 1 introduces the project background, motivation, problem statement, objectives, and scope. Chapter 2 presents the system analysis and design, including the architecture, predictive thermal model, rule-based outfit engine, and user interface. Chapter 3 describes the system implementation, while Chapter 4 discusses experimental results and development challenges. Finally, Chapter 5 concludes the report and outlines future enhancements.

CHAPTER II

SYSTEM ANALYSIS AND DESIGN

2.1. System Overview

FitCast is designed as a multi-stage intelligence pipeline that converts real-time weather data into personalized outfit recommendations. Instead of producing a static result, the system processes information through three sequential intelligence layers.

First, Predictive Intelligence estimates the user's thermal needs using a machine learning model and current weather data. Next, Logic-Based Intelligence applies contextual constraints such as activity type and modesty requirements to ensure the outfit is suitable for the situation. Finally, Adaptive Intelligence refines recommendations over time by learning from user feedback. This layered structure keeps the decision process clear while allowing thermal comfort, context, and personalization to be handled independently.

2.2. System Architecture

The architecture of FitCast is built on a modular design that separates the complex logic from the mobile interface. By keeping these parts independent, the system becomes easier to manage and more accurate. This structure allows backend components—like the Predictive Thermal Model and the Outfit Engine—to be updated or improved without needing to change anything on the user's app.

2.2.1. Architectural Style

FitCast adopts a pipeline-based architectural style in which data is processed sequentially through a series of distinct stages. Each stage refines the output of the previous one, progressively transforming raw environmental inputs into a context-aware outfit recommendation.

This architecture enables clear separation of concerns and high modularity, allowing individual components—such as predictive analysis and decision logic—to be modified independently without affecting the user interface or overall system behavior.

2.2.2. Core System Components

To support the pipeline-oriented flow, FitCast is organized into five core modules, each responsible for a specific stage in transforming environmental data into a final outfit recommendation:

- 1. Environmental Data Processing Module:**

Serves as the entry point of the pipeline, responsible for collecting and normalizing raw meteorological inputs such as temperature, humidity, and wind conditions.

- 2. Predictive Thermal Model:**

Acts as the analytical core of the system, utilizing a machine learning–based prediction mechanism to map environmental conditions and user sensitivity into a discrete thermal layer index representing the required clothing insulation level.

- 3. Adaptive Personalization Module:**

Adjusts system behavior over time by incorporating user feedback, enabling recommendations to align with individual comfort sensitivities.

4. Rule-Based Outfit Engine:

Applies contextual and social constraints, including activity-specific rules and cultural considerations, to ensure the recommendation remains practical and appropriate.

5. Presentation and Interaction Layer:

Represents the final stage of the pipeline, presenting recommendations through a visual interface that integrates avatars and weather-responsive environments.

2.2.3. System Architecture Visualization

Figure 2.1 presents a high-level overview of the FitCast system architecture, illustrating the interaction between the mobile client, external services, and the data-driven intelligence layer. Environmental data and user inputs are processed by a predictive thermal model and refined by a rule-based outfit engine before being displayed to the user through the mobile interface.

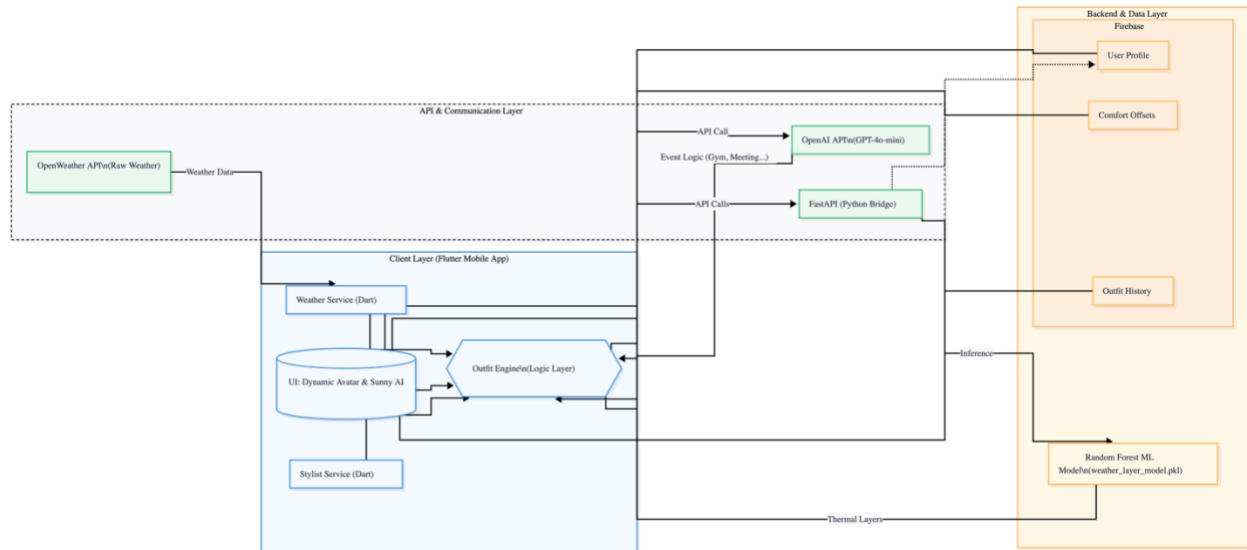


Figure 2.1: High-level system architecture of FitCast illustrating the interaction between the mobile client, external APIs, and the data-driven intelligence pipeline.

2.3. Functional and Non-Functional Requirements

This section outlines the specific operational goals and quality constraints of the FitCast system. These requirements ensure that the architectural modules described in Section 2.2 deliver a reliable, adaptive, and personalized user experience.

2.3.1. Functional Requirements

1. **User Authentication:** The system shall provide secure user registration and login using Firebase Authentication, ensuring persistent user profiles across sessions.
2. **Natural Language Understanding:** The system shall analyze unstructured user input using generative AI to extract relevant activity and clothing intent.
3. **Environmental Data Acquisition:** The system shall retrieve real-time weather parameters, including temperature, wind speed, humidity, cloud coverage, and precipitation.
4. **Thermal Layer Prediction:** The system shall compute perceived temperature using bioclimatic formulas and map environmental features to a discrete clothing layer index (1–5) via a machine learning classifier.
5. **Contextual Rule Enforcement:** The system shall apply rule-based constraints to ensure cultural appropriateness and activity-specific outfit adaptation.
6. **Personalized Feedback Learning:** The system shall allow users to submit thermal comfort feedback, which updates a personalized comfort offset stored in the cloud database.

7. **Dynamic Visualization:** The system shall visually reflect current weather conditions and outfit recommendations using animated backgrounds and a generative avatar.

2.3.2. Non-Functional Requirements

1. **Real-Time Responsiveness:** The system shall generate outfit recommendations with a target latency of under 3 seconds to support real-time user interaction.
2. **Numerical Stability:** The system shall apply input stabilization thresholds to ensure reliable machine learning inference.
3. **Modular Scalability:** The system shall maintain separation between the Flutter client and the FastAPI-based backend to allow independent system updates.
4. **Data Reliability:** The system shall ensure real-time synchronization of user data using a cloud-based NoSQL database.

2.3.3. System Use Case Diagram

The functional interactions between the user, the FitCast system, and external services are visualized in the use case diagram below. This model highlights the dependencies between internal logic and external APIs such as OpenWeather and OpenAI.

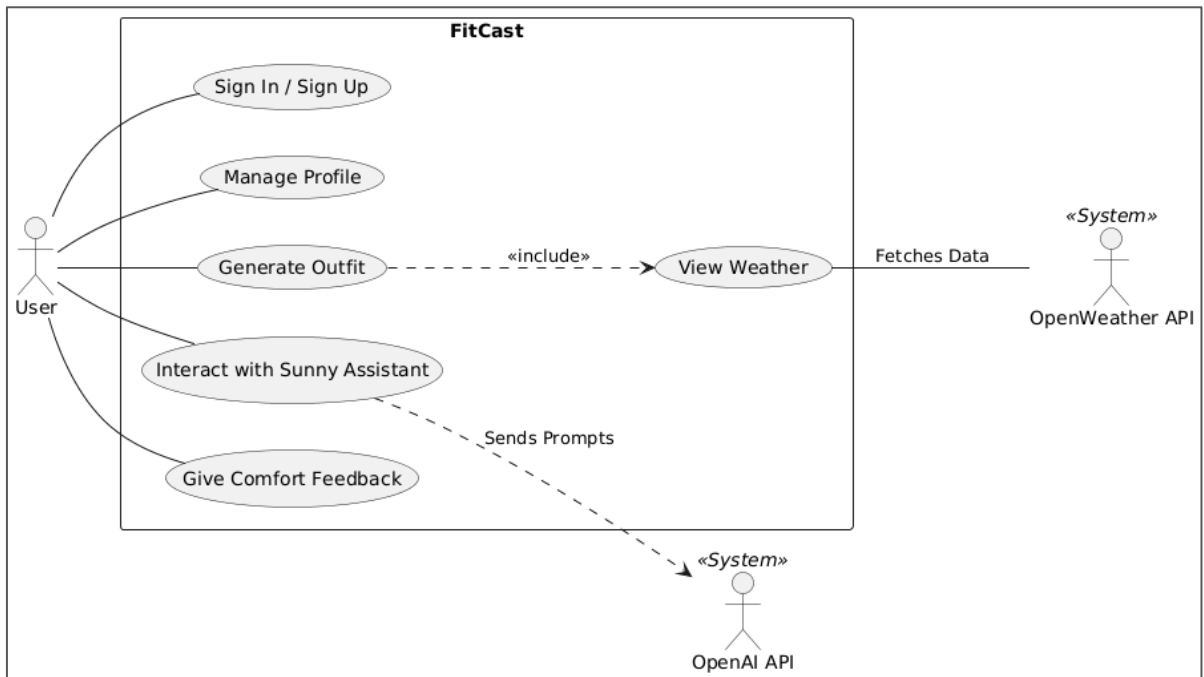


Figure 2.2: Use Case Diagram illustrating the primary interactions between the user, the FitCast system, and external APIs (OpenAI and OpenWeather).

2.3.4. System Sequence Diagram

The sequence diagram below shows the step-by-step process of generating an outfit. It illustrates how the mobile application acts as the main controller for the system. The app handles the conversation with the user, uses OpenAI to understand their intent, and fetches live weather data from OpenWeather. After getting a thermal prediction from the backend (FastAPI), the app combines all this information to decide on the final outfit recommendation locally.

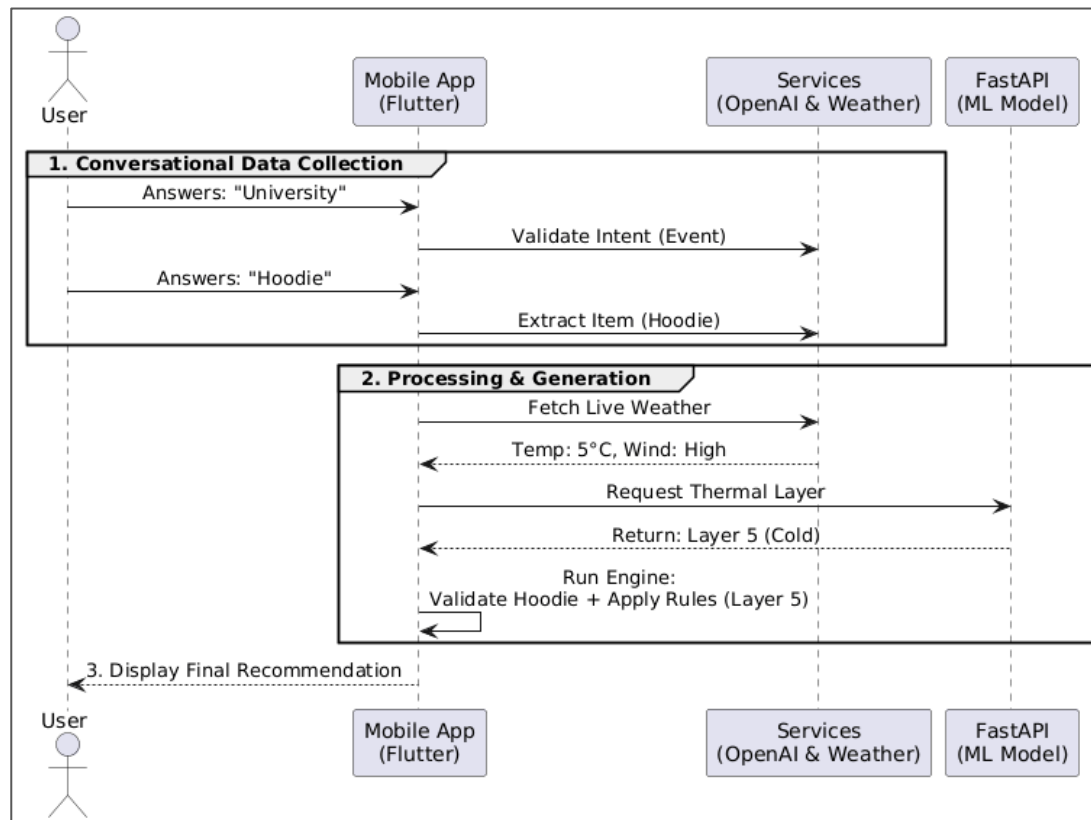


Figure 2.3: Sequence diagram illustrating the conversational data collection and the hybrid processing flow, where the mobile client orchestrates external services (NLU, Weather) and the internal logic engine.

2.4. Predictive Thermal Model

The Predictive Thermal Model serves as the analytical core of the FitCast system. It is responsible for translating multi-dimensional environmental data into a human-centric Thermal Layer Index (0–5) that guides clothing recommendations. This process is divided into two primary stages: bioclimatic feature engineering and machine learning classification.

2.4.1. Bioclimatic Feature Engineering

To model human thermal comfort more realistically, the system computes a Perceived Temperature (PT) value. Unlike ambient temperature alone, PT captures the combined effects of environmental conditions and individual sensitivity on how weather is perceived. Depending on atmospheric thresholds, appropriate bioclimatic formulas are dynamically selected and applied.

The selected formulas are based on established meteorological models commonly used by organizations such as Environment Canada, NOAA, and the Australian Bureau of Meteorology to estimate human thermal perception.

Temperature thresholds follow commonly used ranges associated with standard wind chill and heat index formulations.

Wind Chill Integration ($T \leq 12^{\circ}\text{C}$)

For colder conditions, the system applies a Wind Chill formula that accounts for wind speed to better represent how cold the weather feels.

$$PT = 13.12 + 0.6215T - 11.37(V^{0.16}) + 0.3965T(V^{0.16})$$

where T represents air temperature ($^{\circ}\text{C}$) and V represents wind velocity (km/h).

Heat Index Adjustment ($T \geq 26^{\circ}\text{C}$)

For warmer conditions, the system applies a Heat Index formula that considers humidity to better represent how hot the weather feels.

Atmospheric and Personal Adjustment Logic

To make the perceived temperature more realistic, additional environmental and user-related factors are applied. Heavy rainfall reduces the perceived temperature by up to -3°C to represent cooling effects on clothing, while strong sunlight increases it by up to $+3^{\circ}\text{C}$ under high UV conditions.

A user-defined sensitivity offset ($\pm 2^{\circ}\text{C}$) is also included to reflect individual differences in thermal comfort. This allows the system to slightly personalize the perceived temperature before it is passed to the prediction model.

2.4.2. Random Forest Classification

The engineered features—including perceived temperature, ambient temperature, wind speed, humidity, precipitation intensity, and user thermal sensitivity—are used as inputs to a Random Forest classifier. The model predicts a discrete Thermal Layer Index (0–5), which indicates the amount of clothing insulation needed under the given weather conditions.

A Random Forest approach was selected because it handles non-linear relationships between weather factors effectively and performs well with noisy meteorological data. The model was trained using a synthetically generated dataset that represents a wide range of realistic weather situations, helping the system perform reliably across different conditions.

The final model is built using 200 decision trees with a maximum depth of 12. This setup was chosen to keep the predictions accurate without making the model overly complex. Once the thermal layer is predicted, it is passed to the Outfit Engine, where additional context and rule-based logic are applied to produce the final outfit recommendation.

2.5. Rule-Based Outfit Engine

The Rule-Based Outfit Engine is the final step in the FitCast pipeline. Its role is to take the predicted Thermal Layer Index and turn it into a complete outfit that fits the user's situation. This is done through three main filtering stages.

2.5.1. Context-Aware Activity Filtering

The outfit generation process starts by checking what type of activity or destination the user has selected. Based on this, event-based rules are used to limit the clothing options so the outfit fits the situation.

- **Activity and Destination Matching:**

Clothing categories are filtered according to the selected event to make sure the outfit is suitable for where the user is going.

- **Formal and Structured Settings:**

For places like meetings or office environments, casual items are removed so the recommendation stays appropriate and presentable.

- **Fallback Handling:**

If an event does not have specific rules, or if some information is missing, the system falls back to a general category such as casual or university, based on the user's style preferences.

- **Decision Tracking:**

During this step, the engine keeps track of important decisions, such as why certain items were excluded or user requests were adjusted. This information is later used to provide simple explanations to the user.

2.5.2. Style Compatibility and Anchoring

After filtering by activity, the engine focuses on making sure the outfit looks visually consistent. This is done using a simple anchor-based approach.

- **Anchor Item:**

The engine first selects a main clothing item, usually a bottom such as jeans or a skirt. This item acts as the base for building the rest of the outfit.

- **Matching Logic:**

Once the anchor is chosen, the engine looks for tops and shoes that work well with it. These combinations are ranked as Strong, Medium, or Weak matches, and the stronger matches are preferred to keep the outfit coordinated.

2.5.3. Personalization and Health Integration

The final pass adjusts the outfit to match the user's personal profile and safety requirements:

- **Modesty Logic Tiers:** The rule-based system applies different constraints based on the user's selected modesty level. For female users, this includes distinct logic for Hijabi Modest, Non-Hijabi Modest, and Non-Modest profiles, which automatically control item lengths and accessory requirements like the Hijab. For male users, the system includes Modest and Non-Modest options to manage constraints like the disallowance of shorts.

- **Health and Safety Logic:** The system monitors environmental triggers like high wind or dust. For users with asthma or allergies, the engine injects a protective mask into the recommendation.
- **Variety Mechanism:** To keep suggestions fresh, the engine applies a 90% variety weight, ensuring that items suggested in the most recent outfit are less likely to appear the next day.

CHAPTER III

SYSTEM IMPLEMENTATION

3.1. Development Environment

The development of FitCast was carried out in a local development environment, with a focus on building a cross-platform mobile application and ensuring smooth communication with backend services. The system was designed to support real-time data exchange while keeping the different components independent and easy to manage.

3.1.1. Technology Stack

A set of modern tools and frameworks was selected to support the modular design of the system:

- **Flutter (Dart):** Used to build the cross-platform mobile application. Flutter allows the same codebase to run on both Android and iOS and supports smooth and responsive user interfaces.
- **Firebase:** Used as the backend service for the application. Firebase Authentication handles user login and account management, while Cloud Firestore stores user-related data such as preferences and comfort profiles.
- **FastAPI (Python):** Used to host the machine learning model and expose it through RESTful APIs. This allows the Flutter application to request predictions from the Random Forest model in a lightweight and efficient way.
- **Figma:** Used for designing the user interface and organizing visual assets, including clothing items and avatar layouts used in the application.

3.1.2. Visual Asset Design

Instead of using ready-made or stock images, all visual assets in FitCast were created specifically for the application. This was done to keep the visuals consistent with the system logic and to ensure a clear and unified user experience.

Clothing Asset Creation and Avatar Design

Both the clothing items and the avatar were created using the same design workflow to ensure visual consistency and smooth integration within the system. Initial visuals were generated and refined through several iterations until the desired style and proportions were achieved. The assets were then converted from PNG to SVG format to allow accurate resizing and manual adjustments.

Figma was used extensively to refine outlines, colors, and proportions, and to prepare each element as an individual component. The avatar was designed in a modular way, with separate visual parts for the head, upper body, lower body, footwear, and additional layers.

Each clothing item was exported individually using a fixed frame size to maintain consistent positioning across outfits. These assets were then integrated into the application and dynamically assembled using a custom Avatar Builder component, which layers clothing items based on the system's generated recommendations.

This modular design allows the avatar to clearly reflect different outfit combinations while maintaining consistent proportions and visual structure.

3.2. External API Integrations

The functionality and real-time usefulness of FitCast depend on receiving data from external services in a reliable way. This section explains how weather data is retrieved and how user input is processed through the Natural Language Understanding (NLU) component.

3.2.1. Meteorological Data (OpenWeather API)

To compute realistic perceived temperature values, FitCast retrieves live weather data using the OpenWeather API. Beyond simple data retrieval, the system prepares clean and stable inputs for the machine learning model.

Raw weather responses are processed through a dedicated service layer where missing values are handled safely and numerical inputs are kept within reasonable ranges. Before being passed to the predictive model, key weather features are extracted and normalized. For example, wind speed is stabilized to avoid calculation issues, and precipitation data is combined into a single intensity metric.

Listing 3-1: Core weather feature extraction and wind speed stabilization logic.

```
// Source: lib/services/WeatherService.dart

Map<String, dynamic> extractWeatherFeatures(Map<String, dynamic> data) {
  final double rainMm = data["rain"]?["1h"]?.toDouble() ?? 0.0;
  final double snowMm = data["snow"]?["1h"]?.toDouble() ?? 0.0;

  final double rawWindSpeed = data["wind"]?["speed"]?.toDouble() ?? 0.0;
  // Stabilization: prevent 0.0 wind speed from causing division errors
  final double windSpeed = rawWindSpeed < 0.5 ? 0.5 : rawWindSpeed;

  return {
    "temp": data["main"]["temp"].toDouble(),
    "wind": windSpeed,
    "humidity": data["main"]["humidity"].toDouble(),
    "cloud": data["clouds"]["all"].toDouble(),
    "rain_intensity": _rainIntensityFromMm(rainMm + snowMm),
    "rain_type": _mapRainType(data["weather"][0]["id"]),
  };
}
```

Only the most relevant section of the implementation is shown for clarity.

3.2.2. Intent Extraction and Validation (OpenAI ChatGPT API)

FitCast uses the OpenAI ChatGPT API to understand user input written in natural language. Instead of forcing users to choose from fixed options, the system allows them to freely describe their plans and clothing preferences.

To keep this input reliable and easy to manage, the AI functionality is divided into two separate services, each with a specific purpose. One service handles styling-related intent, such as identifying the user's activity or requested clothing items. The other service focuses on validating user input, making sure the provided information is meaningful before it is saved or used by the system.

This separation helps reduce errors and prevents unclear or invalid input from affecting the overall outfit recommendation process.

Activity and Clothing Intent Extraction (StylistAIService)

The StylistAIService handles activity and intent extraction from natural language user input. Instead of relying on fixed options, users can describe where they are going in their own words.

The service maps these descriptions to a predefined set of event categories defined in the event rules JSON files. If the user input does not exactly match an existing rule, the system maps it to the closest valid event category. This fallback behavior ensures that the rule-based outfit engine always receives a valid and usable event type.

To achieve this, the system uses a carefully designed prompt that enforces strict output formatting and controlled event mapping, while still allowing flexibility in how users describe their plans.

To support different types of user input, the service uses three prompt modes, each responsible for a specific task:

1. activity classification.
2. clothing item extraction.
3. and user-facing explanation generation.

This separation keeps the system flexible while maintaining strict control over the data passed to later stages of the pipeline.

Listing 3-2: Core prompt logic used for activity classification in the NLU layer

```
// Source: lib/services/StylistAIService.dart

if (type == "outfit_plan") {
  systemPrompt = ""
  You are an Event Classifier.
  Map the user's input to ONE valid event key:
  [gym, office, university, airport, dinner, meeting, party, casual,
  formal, comfy].
  Return the result in strict JSON format.
  "";
}
```

Only the most relevant section of the implementation is shown for clarity.

In addition to activity classification, the `StylistAIService` supports two other prompt types. The item extraction prompt maps user-mentioned clothing items to predefined inventory keys, while the explanation prompt generates short, user-friendly messages when certain outfit choices are rejected due to weather or activity constraints.

3.3. Internal Machine Learning Service (FastAPI)

This section describes how the machine learning model was integrated into the system using a FastAPI backend service. To keep the mobile application lightweight, all computationally heavy tasks related to model execution are handled on the server side. The FastAPI service is responsible for managing the trained model and responding to real-time prediction requests from the mobile application.

3.3.1. Feature Engineering and Model Training

The machine learning logic was implemented in Python using the scikit-learn library. Since no suitable real-world dataset existed for the problem domain, a synthetic dataset was generated based on weather conditions and user sensitivity values. This approach allowed the model to cover a wide range of realistic scenarios, including cases such as high temperatures combined with strong wind or rainfall.

For each generated data sample, a perceived temperature value was calculated using bioclimatic formulas. This value was then mapped to a discrete clothing layer, which was used as the target output during model training.

```
if temp <= 12:
    base = wind_chill(temp, wind)
    universal_wind_cooling = 0
elif temp >= 26:
    base = heat_index(temp, humidity)
    universal_wind_cooling = 0.5 * math.sqrt(wind)
else:
    base = temp
    universal_wind_cooling = 0.7 * math.sqrt(wind)

adjusted = base - sensitivity

perceived = (
    adjusted
    + rain_effect
    + uv_effect
    - universal_wind_cooling
    + extreme_cold_boost
    + extreme_heat_boost
)

return perceived
```

Figure 3.1: illustrates the core logic used to compute perceived temperature during dataset generation.

3.3.2. Model Deployment (FastAPI Endpoint)

To make the trained machine learning model accessible to the mobile application, it was deployed as a RESTful web service using FastAPI. Hosting the model in a separate backend service allows the mobile application to remain lightweight while offloading computational tasks to the server.

The model is loaded once when the server starts, which improves efficiency when handling multiple prediction requests. Before performing inference, incoming feature data is converted into a structured format and aligned with the feature schema used during training. This step ensures consistency between training and inference and prevents errors caused by missing or reordered features.

```
@app.post("/predict-layer")
def predict_layer(data: dict):
    df = pd.DataFrame([data])
    df = pd.get_dummies(df, columns=["rain_type"], prefix="rain")
    df = df.reindex(
        columns=model.feature_names_in_,
        fill_value=0
    )

    layer = int(model.predict(df)[0])

    return {"layer": layer}
```

Figure 3.2: FastAPI inference endpoint used to predict the clothing layer from processed weather features.

3.3.3. Client-Side Integration (Flutter)

The mobile application communicates with the FastAPI backend using a dedicated service class in Flutter. Weather and user-related features are converted into JSON format and sent to the prediction endpoint using an HTTP POST request.

The backend returns a predicted clothing layer value (0–5), which is then used by the outfit engine. This value determines which clothing combinations are selected based on the current thermal conditions.

```
Future<int> predictLayer(Map<String, dynamic> features) async {  
    final url = Uri.parse("$_baseUrl/predict-layer");  
  
    final response = await http.post(  
        url,  
        headers: {  
            "Content-Type": "application/json",  
        },  
        body: jsonEncode(features),  
    );  
}
```

Figure 3.3: Flutter service sending feature data to the FastAPI prediction endpoint.

3.4. Mobile Application Implementation

The mobile application acts as the main point where all system components come together. It manages the user session, applies the rule-based outfit logic, and displays the final outfit through a dynamic and interactive visual interface.

3.4.1. Rule-Based Engine Execution

The rule-based outfit selection is executed on the mobile device through the `generateSmartOutfit` method. This method applies the filtering pipeline described in Chapter II and is responsible for converting the predicted thermal layer into a complete outfit recommendation.

At the start of execution, the engine loads the required rule definitions from structured JSON files, including event-based rules and compatibility constraints. These rules are used to limit available clothing categories according to the selected activity or destination.

The outfit selection follows a hierarchical process based on an anchoring mechanism. In this approach, a bottom item (such as jeans or a skirt) is selected first and treated as the anchor of the outfit. Once the anchor is chosen, the engine filters and ranks available top items by evaluating their compatibility with the selected bottom. Compatibility levels are defined in the `compatibility_rules.json` file and ensure that only visually and contextually appropriate combinations are considered.

By applying this step-by-step filtering strategy, the engine produces outfit recommendations that are both suitable for the user's context and visually consistent.

Listing 3-3: Implementation of the anchoring logic used to select compatible outfit combinations.

```
// 1. Pick Anchor Item (Bottom-First Strategy)
outfit["bottom"] = _pickBestItem("bottom", allowedCats, ...);

// 2. Compatibility Matching: Filter tops based on anchor strength
if (outfit["bottom"] != null) {
    final anchorData = compatibilityRules?[gender]["anchors"?[outfit["bottom"]]];

    if (anchorData != null) {
        matchedTops = (anchorData["tops"] as List)
            .where((t) => ["strong", "medium"].contains(t["strength"])) // Filter logic
            .map((t) => t["item"].toString()).toList();
    }
}
```

3.4.2. Contextual Adaptation and Safety Logic

In addition to basic style matching, the mobile application applies contextual and safety-related logic to handle cases where weather conditions, user feedback, and activity requirements may conflict. This ensures that the final recommendation focuses on comfort and safety, rather than appearance alone.

Adaptive Thermal Adjustment (Feedback Offset)

To support the feedback loop described in the system design, the outfit engine uses a thermal offset based on the user's previous feedback. When a user repeatedly reports feeling too cold or too warm, this value is applied before the outfit selection starts.

For instance, if a user often reports feeling cold, the engine slightly increases the selected thermal layer before choosing clothing items. This allows the system to slowly adjust to the user's personal comfort level over time, without changing the core prediction model or the predefined rules.

Commute and Activity Resolution Logic

One challenge addressed in the system design is the difference between outdoor weather conditions and indoor activity requirements, such as walking to the gym in cold weather and exercising indoors. To handle this, the engine evaluates two thermal values: an activity layer constrained by event rules to prevent overheating, and an outdoor layer based on actual weather conditions.

If the difference between these values exceeds a defined threshold, a commute rule is applied. In this case, heavy outerwear is added for the commute, while the base outfit remains lightweight and suitable for the activity.

Health and Safety Prioritization

The engine cross-references environmental conditions such as wind speed or dust levels with the user's medical profile, and if potentially harmful conditions are detected for users with allergies or respiratory sensitivities, the system automatically includes protective accessories, such as a mask. This also acts as a reminder for the user to take appropriate health precautions when facing unfavorable environmental conditions.

3.4.3. Dynamic Avatar Visualization

This section explains how the outfit recommendation generated by the engine is visually presented to the user. A custom AvatarBuilder component was implemented to separate avatar rendering from the outfit suggestion screen and maintain a modular design. The component displays the outfit on a user avatar by layering SVG assets using a Stack widget.

Clothing items are rendered in a fixed order to ensure correct visual layering, with accessories such as masks or hijabs displayed on top. This approach allows the avatar to adapt dynamically to different outfit combinations while maintaining consistent proportions and visual clarity.

CHAPTER IV

EXPERIMENTAL RESULTS AND EVALUATION

4.1. Introduction

This chapter evaluates the FitCast system by presenting key user interface screens and scenario-based examples. The evaluation focuses on how users interact with the application and how the system responds to different preferences and environmental conditions. Emphasis is placed on usability, clarity of interaction, and the system's ability to translate complex decision logic into an intuitive user experience.

4.2. User Interface Implementation

The user interface is organized into a set of main screens that guide the user through account access, profile setup, and daily interaction with the application. The design focuses on keeping interactions simple and easy to follow, while clearly showing the results of user choices. In the following sections, key interface screens are presented to demonstrate how users interact with FitCast in real usage scenarios.

These screens form the foundation of the user experience and prepare the user for personalized interaction with the system.

4.2.1. Authentication and Account Access

Before accessing personalized features, users authenticate through a simple sign-in and sign-up interface using Firebase Authentication. This allows user profiles, preferences, and feedback data to be securely stored and reused across sessions.

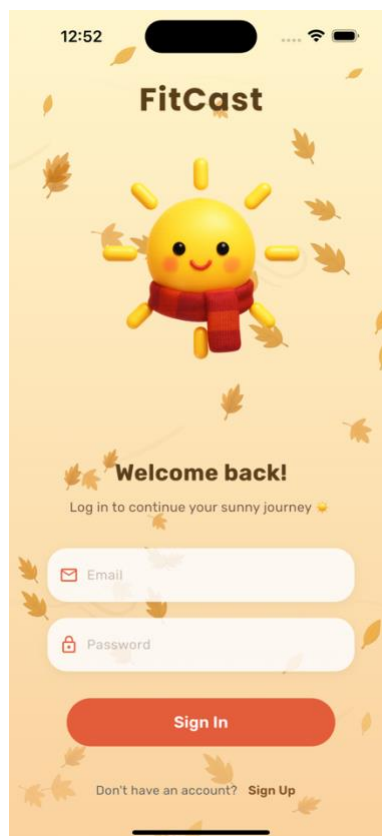


Figure 4.1: FitCast sign-in and sign-up interface used for secure user authentication.

4.2.2. Onboarding and Profile Customization

During first-time sign-up, the Sunny assistant guides users through the initial setup by collecting essential profile information needed for personalized outfit recommendations. This includes preferences related to modesty, thermal sensitivity, and basic health conditions such as allergies or asthma. Users can later revisit this flow from the application menu to update their responses at any time. The collected profile data is stored securely and used by the rule-based outfit engine and personalization logic to adapt recommendations to each user's individual needs.

The Sunny assistant interaction is implemented using the natural language processing approach described earlier in Chapter III.

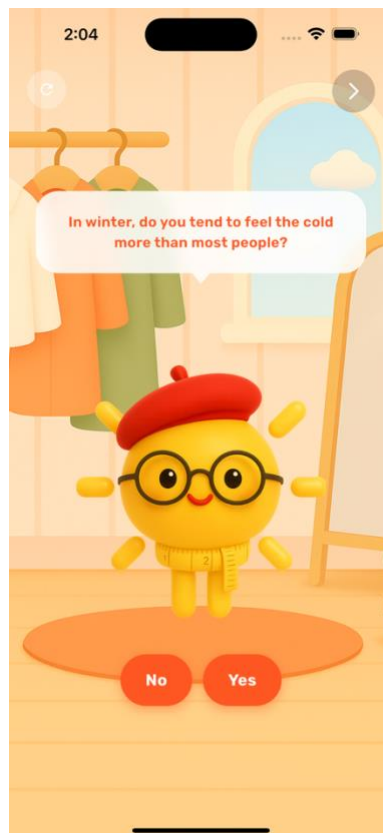


Figure 4.2: Sunny assistant collecting user thermal sensitivity preferences during the initial profile setup.

4.2.3. Main Dashboard and User Interaction

The main dashboard serves as the primary screen for user interaction. It displays a dynamic weather background that reflects real-time conditions such as rain, snow, or clear skies, and adapts to the time of day through sunrise, daytime, sunset, and night scenes.

At the bottom of the screen, a text input field allows users to communicate with the system using natural language (e.g., “I’m going to university”). An on-screen avatar representing the user (male or female) visually reflects the generated outfit. The system first asks about the intended activity and then prompts the user to specify any preferred clothing item, using this information to generate an outfit recommendation that respects weather conditions and event constraints.



Figure 4.3: Main dashboard with dynamic weather visualization and avatar-based outfit interaction.

4.2.4. Dynamic Outfit Visualization and Layer Control

After an outfit is generated, the interface transitions to the outfit visualization screen. The recommended clothing items are displayed on a customizable avatar, allowing users to clearly see how the outfit would appear when worn. This screen focuses on helping users understand both the visible outfit and the functional layers selected by the system.

To support layered clothing recommendations, the interface provides a layer control mechanism that allows users to switch between different clothing views. These views include the base layer (such as thermal wear and socks), the main outfit layer (top and bottom), and the outer layer (jackets and accessories used outdoors). This separation helps users understand how the system adapts clothing choices for indoor and outdoor conditions.



Figure 4.4: Outfit visualization screen displaying the outerwear layer, including jackets and accessories, with layer control arrows for navigating between clothing layers.

In addition, the screen includes thermal feedback options that allow users to indicate whether the suggested outfit felt too cold, comfortable, or too warm after use. This feedback is stored and later used to adjust future outfit recommendations, enabling gradual personalization based on real user experience.



Figure 4.5: Thermal feedback interface allowing users to rate the recommended outfit as too cold, comfortable, or too warm for adaptive personalization.

Layer navigation and thermal feedback are presented in separate steps, allowing users to first review the outfit layers and later provide feedback based on their actual comfort.

4.3. System Scenarios and Case Studies

To validate the system's intelligence, several user scenarios were tested. These cases show how FitCast adapts its recommendations based on user needs while keeping the same weather conditions.

Scenario A: Thermal Sensitivity Adjustment

Conditions: Effective Layer 5 (very cold and windy), Activity: *University*.

Test Case: Two users were evaluated under the same weather and activity conditions but with different thermal sensitivity profiles.

User A (Cold-Sensitive):

The system increased the effective thermal layer and recommended additional insulation. The generated outfit included a thermal top, thermal leggings, and a puffer jacket to provide sufficient warmth.

User B (Heat-Sensitive):

The system slightly reduced the effective thermal layer. While essential outerwear was still recommended for outdoor safety, heavier internal layers were replaced with lighter base layers to improve comfort.

Observation:

Although the internal layering differs between the two users, the overall outfit structure remains similar. Both users receive appropriate outerwear to handle outdoor conditions, while personalization mainly affects comfort-related layers.

System Behavior Explanation:

This behavior shows that FitCast applies thermal personalization in a controlled manner. Adjustments are made without causing drastic or unrealistic outfit changes, ensuring thermal safety, consistency, and stable recommendations.

Scenario B: Machine Learning–Based Comfort Adaptation

Conditions: Mild–Cold Weather (Layer 4), Clear Sky , Activity: “Cafe”

Test Case: The scenario evaluates the effect of historical comfort feedback on outfit generation. Two executions were performed under identical environmental and activity conditions, differing only in the applied machine learning comfort offset.

Baseline Case (No ML Adjustment):

With no prior comfort feedback applied (ML offset = 0), the system selected a balanced outfit appropriate for both indoor comfort and outdoor conditions. The recommendation included a light sweater, standard trousers, a trench coat for the commute, and seasonal accessories. A light base layer was added to maintain thermal comfort without excessive insulation.

ML-Adjusted Case (Warm Sensitivity Detected):

After registering repeated feedback indicating that the user tends to feel warm, a negative ML offset was applied (ML offset = -0.85). The system reduced the effective thermal layer by one level, resulting in the removal of internal base layers. However, a soft constraint was enforced to preserve essential outerwear for outdoor safety. As a result, the final outfit maintained a suitable coat for the commute while simplifying internal layering to improve comfort.

Observation:

The machine learning module influenced the outfit selection in a controlled manner, primarily affecting comfort-related base layers while preserving protective outerwear. The overall outfit structure remained consistent, avoiding abrupt or unsafe changes.

System Behavior Explanation:

This scenario shows that FitCast applies machine learning feedback gradually. Adjustments mainly affect base layers, while essential outerwear is kept for safety. This ensures personalization without extreme or unrealistic outfit changes.

In addition to the core interaction screens presented above, the application includes supporting interfaces such as a weather overview screen, an outfit history screen, and a favorites screen. These views allow users to review forecast details, revisit previously generated outfits, and save preferred recommendations. Since these screens do not influence the outfit generation logic, they are not included in the evaluation visuals of this chapter.

4.4. Implementation Challenges and Solutions

During the development of FitCast, several logical challenges were encountered while combining weather data, user preferences, and adaptive feedback. This section presents the three most significant challenges and the solutions applied to address them.

4.4.1. Commute vs. Destination Conflict

Problem:

In cold weather, the system faced a conflict between outdoor conditions and indoor destinations. For example, when the user was going to university, indoor rules limited the outfit to lighter layers, while outdoor weather required heavier clothing. This caused either indoor overheating or insufficient protection during the commute.

Solution:

The outfit logic was redesigned to separate indoor comfort from outdoor safety. The system now:

- Selects the main outfit based on the destination (indoor activity).
- Adds outerwear based on real outdoor weather conditions.

Result:

Users receive outfits that are comfortable indoors while still being protected during cold or windy commutes.

4.4.2. Robust Handling of Unclear User Input

Problem:

The system originally required exact matches between user input and predefined event rules. If the user entered an unfamiliar activity, the engine could not determine the correct clothing context.

Solution:

A fallback mechanism was implemented:

- User input is mapped to general event categories.
- If no direct match exists, the system falls back to the user's preferred style.

Result:

FitCast can now handle a wide range of user inputs without failing or producing inappropriate outfit recommendations.

4.4.3. Thermal Personalization Without Extreme Changes

Problem:

The system uses both the user's thermal sensitivity (cold-sensitive or heat-sensitive) and past comfort feedback to adjust outfit warmth. When these two factors were applied too strongly at the same time, the system sometimes suggested outfits that were either too heavy or too light for the actual weather.

Solution:

A controlled adjustment approach was implemented. Instead of making large changes, the system now:

- Adjusts mainly internal layers (such as base layers).
- Restricts how much the outfit can change compared to the real weather conditions.

Result:

Users with different thermal sensitivities receive slightly different outfits under the same weather conditions, but the recommendations remain realistic, safe, and comfortable rather than extreme.

4.4.4. Summary

These solutions improved the stability, adaptability, and realism of the outfit recommendation system. By resolving conflicts between environment, behavior, and user intent, FitCast delivers personalized recommendations while maintaining safety and consistency.

CHAPTER V

CONCLUSION AND FUTURE WORK

5.1. Conclusion

The FitCast project successfully achieved its main goal of transforming raw weather data into practical and personalized outfit recommendations. Rather than relying on generic forecasts, the system considers user comfort, daily activities, and environmental conditions to support better clothing decisions in real-life situations.

During development, the project showed that combining a machine learning model with a rule-based outfit engine is an effective approach. The system can handle realistic challenges, such as balancing cold outdoor weather with indoor activities, by separating indoor comfort from outdoor safety through layered logic. The feedback mechanism also allows FitCast to improve over time by learning from user responses instead of depending only on initial profile settings.

Several challenges were faced, especially with overlapping rules, understanding user input, and avoiding extreme outfit changes. These were addressed by applying clearer priority logic, simple semantic input handling, and limits on how much recommendations can change. As a result, the system became more stable, easier to understand, and more focused on the user.

Overall, FitCast demonstrates how intelligent system design can enhance everyday decision-making. By combining weather awareness, personalization, and adaptive behavior, the application delivers outfit recommendations that are practical, comfortable, and safe for daily use.

5.2. Future Work

While the current implementation of FitCast satisfies its core functional objectives, several technical extensions could further enhance the system's accuracy, automation, and personalization capabilities.

Wardrobe Digitization using Computer Vision:

Currently, FitCast recommends clothing at a category level rather than specific user-owned items. A future enhancement could allow users to upload photos of their wardrobe, which would be processed using computer vision techniques to classify items by type and material. This would enable the outfit engine to recommend pieces the user actually owns, increasing practicality and personalization.

Calendar API Integration:

Integrating FitCast with external calendar services such as Google Calendar or iOS Calendar would allow the system to proactively generate outfit recommendations. By combining scheduled events with weather forecasts, the application could prepare outfits in advance, reducing manual input and supporting automated daily planning.

Real-Time Biometric Integration:

Future versions of FitCast could incorporate data from wearable devices such as smartwatches. Metrics like heart rate or estimated body temperature could be used as real-time physiological indicators. This data could be fed into the thermal adjustment logic, allowing the system to adapt recommendations dynamically based on the user's physical state rather than relying solely on subjective feedback.

REFERENCES

- [1] OpenAI, 2024, *Chat Completions API Documentation*, OpenAI Platform, viewed on 5 January 2026, <https://platform.openai.com/docs/guides/chat>.
- [2] OpenWeather, 2024, *Current Weather Data API*, OpenWeatherMap, viewed on 5 January 2026, <https://openweathermap.org/current>.
- [3] Flutter, 2024, *Flutter Architectural Overview*, Google Developers, viewed on 5 January 2026, <https://docs.flutter.dev/resources/architectural-overview>.

Software and Design Tools

- [4] Google, 2024, *Google Colaboratory (Colab)*, Google Research, viewed on 20 December 2025, <https://colab.research.google.com/>. (*Used for training the ML model*).
- [5] Figma, 2024, *Figma Interface Design Tool*, Figma Inc., viewed on 10 December 2025, <https://www.figma.com/>. (*Used for UI/UX design*).
- [6] Canva, 2024, *Canva Visual Suite*, Canva, viewed on 12 December 2025, <https://www.canva.com/>. (*Used for presentation and visual assets*).
- [7] OpenAI, 2024, *DALL·E 3 Image Generation*, OpenAI, viewed on 15 December 2025, <https://openai.com/dall-e-3>. (*Used for generating avatar assets*).
- [8] PNG2SVG, 2024, *Online Image Vectorizer*, PNG2SVG.com, viewed on 18 December 2025, <https://png2svg.com/>. (*Used for converting assets to SVG for the avatar*).